UNIT 1 · LESSON 5 · CASE 1

The Broken Odometer

ANSWER KEY

Roles (switch each case): **Decoder** works the bits · **Checker** verifies every step · **Runner** brings the checkpoint to the teacher.

1	1	1	1	+1 →	1	?	?	?	?

A tiny computer stores numbers in **exactly 4 bits**. Like a car odometer, it can't add a new digit when it runs out of room.

Vocabulary

`overflow error`
A result is too big to fit in the bits available, so the extra digit is lost and the number wraps around (like 1111 + 1 → 0000).

`round-off error`
A value can't be stored exactly with the bits available, so the computer stores the closest value it can.

Part A: Whole numbers place values 8, 4, 2, 1

1. What is the largest number 4 bits can store?

1	1	1	1	= 15 in decimal

2. The computer holds 1111 and adds 1. What does it hold now?

0	0	0	0	Error name: overflow error *(1111 + 1 = 10000, and the leading 1 has no room)*

Part B: Engineers relabel the bits place values 2, 1, ½, ¼

Place value	2	1	½	¼	Value
3. Store 2.75	1	0	1	1	2.75
4. Store 1.3 (closest you can)	0	1	0	1	1.25
5. Largest value possible	1	1	1	1	3.75

4b. For #4: how far off is your answer from 1.3? 0.05 Error name: round-off error

Check both neighbors: 1.25 is 0.05 away, 1.5 is 0.2 away, so 1.25 is closest.

5b. For #5: what happens if you add 0.5 to the largest value? Why?

Overflow error. 3.75 + 0.5 = 4.25, which needs a 4s place the computer doesn't have. The extra bit is lost, so it wraps around and holds 0001 = 0.25.

CHECKPOINT 1 Bring your answers to **#2** and **#4**.

teacher

UNIT 1 · LESSON 6 · CASE 2

The Secret Message

ANSWER KEY

`H → 72 → 01001000` Every character you type is stored as a **number**, and every number is stored as **bits**. Sender and receiver just have to agree on the code.

Part A: ASCII each capital letter is one 8-bit byte

A	B	C	D	E	F	G	H	I	J	K	L	M
65	66	67	68	69	70	71	72	73	74	75	76	77

N	O	P	Q	R	S	T	U	V	W	X	Y	Z
78	79	80	81	82	83	84	85	86	87	88	89	90

Place value	128	64	32	16	8	4	2	1

1. Decode the message.

`01000010`	`01011001`	`01010100`	`01000101`
# 66 letter B	# 89 letter Y	# 84 letter T	# 69 letter E

*Example: 01000010 = 64 + 2 = 66 = B. The word is **BYTE**.*

Part B: Class code 5 bits each · space = 0, A = 1, B = 2, … Z = 26

2. Decode the message.

`00011`	`01111`	`00100`	`00101`
# 3 letter C	# 15 letter O	# 4 letter D	# 5 letter E

*The word is **CODE**.*

3. Encode your first name in the class code.

Answers vary. Example, SAM: S = 19 → 10011 A = 1 → 00001 M = 13 → 01101

Write the 5 bits for every letter, not the decimal numbers.

4. Why are 5 bits enough for 27 symbols?

5 bits make $2^5 = 32$ different patterns (00000 to 11111, or 0 to 31). 32 is more than 27.

5. Add lowercase letters too (53 symbols). How many bits per symbol now? 6 bits *($2^5 = 32$ is too few; $2^6 = 64$ is enough)*

6. Why can't 8-bit ASCII handle every language and emoji? What does the world use instead?

8 bits only make $2^8 = 256$ codes, but the world's languages and emoji have many thousands of characters. The world uses Unicode, which gives every character its own number. UTF-8 stores each one in 1 to 4 bytes.

CHECKPOINT 2 Bring **both decoded words**.

teacher

UNIT 1 · LESSON 7 · CASE 3

The Hidden Picture

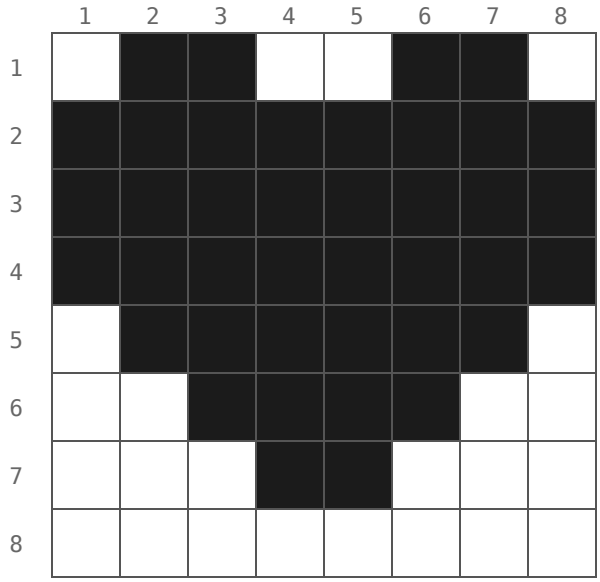
ANSWER KEY

The rules. The first 8 bits are the image's **width**. The next 8 bits are its **height**. Every bit after that is one pixel, row by row, left to right. `1` = black `0` = white

The file

`00001000` `00001000` `01100110` `11111111` `11111111` `11111111` `01111110` `00111100` `00011000` `00000000`

Green bytes are the header (metadata), not pixels.



1. Width = 8

Height = 8

00001000 = 8 for both. Header bytes are not rows.

2. Shade the grid. What is the picture?

A heart

3. How many bits is the whole file, header included?

80 bits (16 header + 64 pixel)

4. The same picture at 16 × 16 pixels: how many pixel bits?

16 × 16 = 256 bits (4× as many; each side doubled)

5. A real sunset is **analog**. Why is any digital image only an approximation of it?

A real sunset changes smoothly, with endless shades and detail. A digital image breaks it into a fixed number of pixels, and each pixel can only be one of a limited number of colors. Anything in between gets rounded off.

CHECKPOINT 3

Bring **the picture** and your answer to **#3**.

teacher

UNIT 1 · LESSON 8 · CASE 4

The Color Lab

ANSWER KEY

R G B Each pixel uses **3 bits**: one for **R**ed, one for **G**reen, one for **B**lue light. A 1 turns that light on. Mixing light makes the other colors.

Color key

000 black	100 red	010 green	001 blue
110 yellow	101 magenta	011 cyan	111 white

The file 4 pixels wide, 2 pixels tall, row by row

`100` `110` `010` `001` `111` `101` `011` `000`

1. Color the grid (or write each color's name).

	1	2	3	4
1	color: **red**	color: **yellow**	color: **green**	color: **blue**
2	color: **white**	color: **magenta**	color: **cyan**	color: **black**

2. How many colors are possible with 3 bits per pixel? **8 (2^3)**

3. How many colors with 2 bits per color (6 bits per pixel)? **64 (2^6, or 4 × 4 × 4)**

With 8 bits per color (24 bits per pixel)? **16,777,216 (2^{24} ≈ 16.8 million)**

4. A 1920 × 1080 photo uses 24 bits per pixel. How many **bytes** is it, before compression? Show your work.

1920 × 1080 = 2,073,600 pixels
24 bits per pixel ÷ 8 = 3 bytes per pixel
2,073,600 pixels × 3 bytes = 6,220,800 bytes (about 6.2 MB)

*File size uses the number of **pixels**, not the number of colors. $2^{24} \div 8$ is not the answer.*

5. Why might an app choose fewer bits per pixel? What does it give up?

Smaller files: less storage and faster to send or load. It gives up color detail, since fewer colors are possible.

CHECKPOINT 4 Bring the colors in **row 2** and your answer to **#4**.

teacher